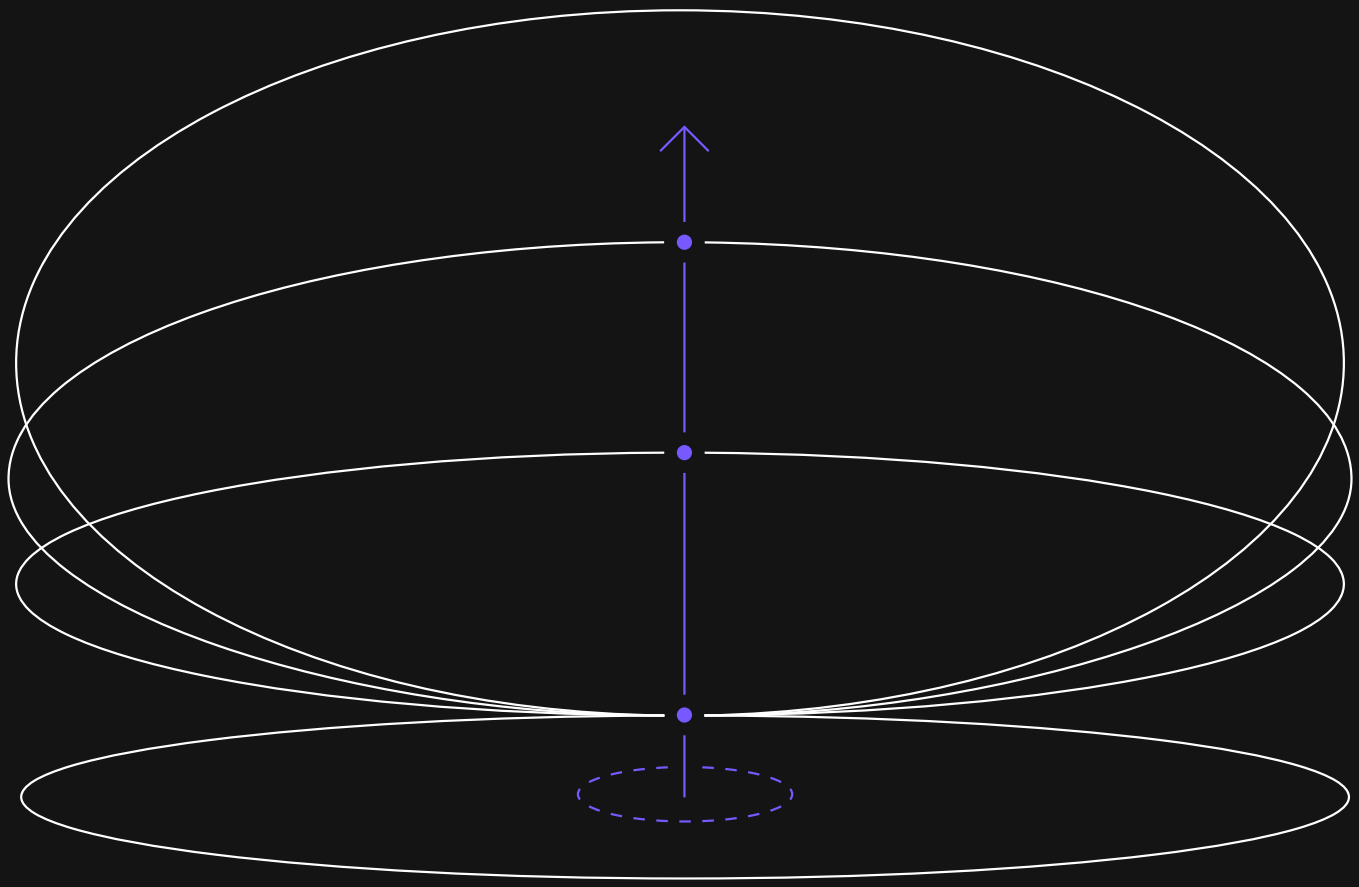
Машинное обучение | Лонгрид

Глубокое обучение | 08

BERT, GPT, T5



Предобученные языковые модели

Предобученная языковая модель получает обучающий сигнал из большого корпуса текстов, а затем используется в прикладной задаче. В этой лекции рассматриваются BERT, GPT и T5. Они различаются доступным контекстом, устройством обучающего примера и правильным ответом на этапе предобучения.

Для каждой модели зададим вход, целевую переменную и функцию потерь, а затем разберем применение к классификации или генерации. Токенизация, таблицы эмбеддингов и блоки трансформера используются как готовые компоненты.

Содержание

1 Предобучение и контекстные представления **2**

2 BERT: восстановление токенов по контексту **2**

3 GPT: предсказание следующего токена **5**

4 T5: условная генерация текста **7**

5 Сравнение моделей и оценка качества **10**

A Контекстные представления: полный разбор **11**

B Разметка последовательности и CRF **13**

C Маскированное обучение на изображениях **18**

Литература **19**

1. Предобучение и контекстные представления

Допустим, мы хотим определять тональность рецензии на фильм. На входе текст, на выходе один из двух классов: положительная или отрицательная рецензия. Можно сразу обучать всю сеть на размеченных рецензиях. Но тогда и значения слов, и связи между ними, и правило классификации придется изучать по одной и той же небольшой выборке.

Вместо этого сначала возьмем большой корпус обычных текстов. Разметки тональности там нет, зато есть сами последовательности слов. Из них можно построить задачу предсказания: восстановить скрытый токен или предсказать следующий. Обучим модель на этой задаче, а затем используем полученные параметры для классификации рецензий. Первый этап называется предобучением, второй - дообучением на прикладную задачу.

После токенизации текст записывается как $x = (x_1, \dots, x_T)$, где x_t - элемент словаря $\mathcal{V}$. Обозначим размер словаря через $V = |\mathcal{V}|$, размер скрытого состояния через d, параметры модели через θ. Отдельные векторы будем записывать столбцами, а матрицу состояний $H \in \mathbb{R}^{T \times d}$ составлять из строк $h_t^\top$. Если в последовательность добавляются специальные токены, они тоже имеют свои позиции и векторные представления.

В статической таблице эмбеддингов одному токену соответствует один вектор. Например, «стекло» в выражении «оконное стекло» и в предложении «масло стекло на пол» получает один и тот же входной эмбеддинг. После обработки последовательности состояние этого токена уже может зависеть от окружающих слов:

$$h_t = F_\theta(x)_t.$$

Здесь F_θ - сеть, которая обрабатывает текст, а индекс t выбирает состояние нужной позиции. Обучаются входные эмбеддинги и параметры самой сети. При применении модели на новом предложении параметры могут быть фиксированы, но состояния вычисляются заново и зависят от этого предложения.

В CoVe контекстные признаки получали из кодировщика модели перевода. В ELMo использовали состояния двух направленных рекуррентных языковых моделей [1, 2]. Общая схема состоит из двух этапов: сначала способ получения признаков обучается на большой задаче, затем эти признаки используются в другой. Полный разбор CoVe и ELMo приведен в приложении.

Качество предобучения и качество классификации проверяются отдельно. Модель может хорошо предсказывать токены и при этом ошибаться в тональности рецензии. Поэтому окончательный результат переноса измеряем на отложенных примерах той задачи, которую действительно собираемся решать.

2. BERT: восстановление токенов по контексту

2.1. Вход модели

Начнем с BERT. Мы хотим получать представление каждого токена с учетом всего предложения. В частности, для слова в середине текста полезны слова и слева, и справа. Поэтому возьмем кодировщик трансформера с двунаправленным self-attention [3].

Здесь возникает ограничение обучающей задачи. Если подать модели исходный токен и попросить предсказать этот же токен, она сможет просто скопировать вход. Нужно сначала убрать часть информации, а затем попросить ее восстановить. Например, исходный текст - «масло стекло на пол», а вход модели - «масло `[MASK]` на пол». Правильный ответ в скрытой позиции - «стекло». Оба соседних фрагмента доступны и могут участвовать в предсказании.

В примерах будем для простоты изображать слова отдельными токенами. На практике исходный BERT использует WordPiece: одно слово может состоять из нескольких подсловных токенов. Маскирование и функция потерь задаются на уровне токенов модели.

Для одного текста вход имеет вид `[CLS]` $x_1, \dots, x_T$ `[SEP]`. Для двух фрагментов:

$$\texttt{[CLS]},\ A,\ \texttt{[SEP]},\ B,\ \texttt{[SEP]}.$$

Токен `[CLS]` дает отдельную позицию, состояние которой можно использовать для классификации всего входа. Токен `[SEP]` отмечает границы фрагментов. Все эти токены входят в последовательность и обрабатываются тем же кодировщиком.

Начальное представление токена в позиции t равно сумме трех векторов:

$$h_t^{(0)} = e(\tilde{x}_t) + p_t + s_{a_t}, \qquad e(\tilde{x}_t), p_t, s_{a_t} \in \mathbb{R}^d.$$

Здесь $\tilde{x}_t$ - токен, который действительно подали на вход после маскирования, e - таблица токенных эмбеддингов, p_t - обучаемое представление позиции, s_{a_t} - представление принадлежности к фрагменту A или B. Эти таблицы обучаются вместе с остальными параметрами BERT. Затем состояния проходят через блоки кодировщика.

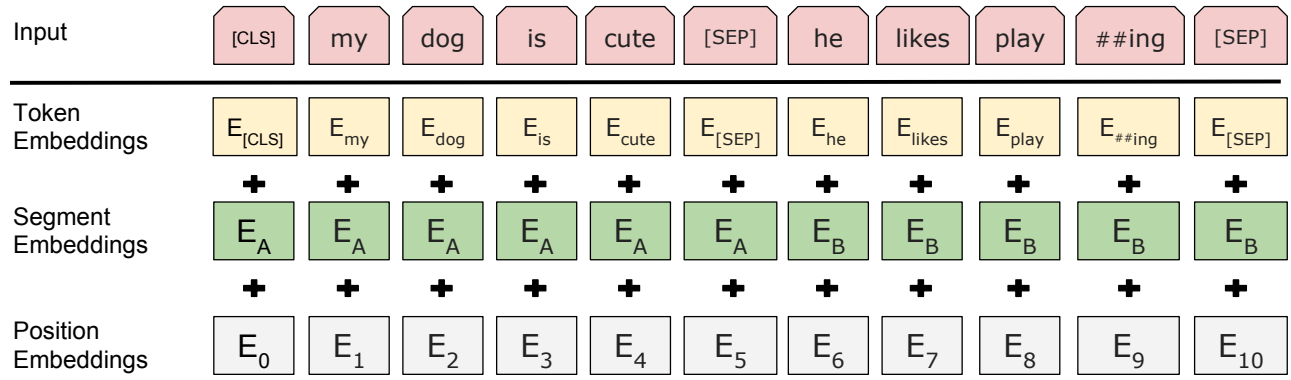


Рис. 1: Входное представление BERT: сумма токенного, сегментного и позиционного эмбеддингов. Devlin et al., рис. 2 [3].

Причинной маски в кодировщике нет. Токен в середине предложения может обращаться к доступным позициям с обеих сторон. Позиции заполнения `[PAD]`, которые добавили для выравнивания длин в батче, исключаются из внимания отдельно.

2.2. Masked language modeling

Выберем множество позиций $\mathcal{M}$, которые требуется восстановить. Исходная последовательность x остается правильным ответом, а измененная последовательность $\tilde{x}$ становится входом. К скрытому состоянию выбранной позиции применяется классификатор по словарю. Он выдает вероятности возможных исходных токенов.

Если правильный токен в позиции t равен x_t, вклад этой позиции в потерю равен $-\log p_\theta(x_t \mid \tilde{x})$. Складываем такие величины:

$$\mathcal{L}_{\text{MLM}}(x, \tilde{x}) = -\sum_{t \in \mathcal{M}} \log p_\theta(x_t \mid \tilde{x}).$$

MLM означает masked language modeling. Индекс t пробегает выбранные для предсказания позиции, а не все позиции входа. Чем больше вероятность правильного токена, тем меньше потеря. Градиент проходит через выходной классификатор, все блоки кодировщика и входные таблицы эмбеддингов.

В исходном BERT выбирают 15% позиций. Среди выбранных 80% заменяют на `[MASK]`, 10% - на случайный токен, а еще 10% оставляют без изменения. Правильным ответом во всех трех случаях остается исходный токен. Например, из 1000 подходящих позиций в среднем выберут 150: 120 получат маску, 15 - случайный токен, 15 останутся прежними. Это ожидаемые количества, а не обязательный состав каждого батча.

Такое правило уменьшает различие между предобучением и последующим использованием, когда специальных масок обычно нет. Нужно различать выбранную для предсказания позицию и фактически скрытую позицию: это разные множества. Потеря считается и для выбранных, но оставленных без изменения токенов.

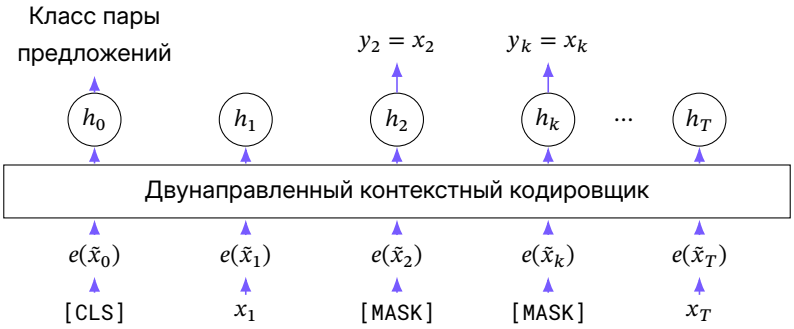


Рис. 2: Восстановление выбранных токенов и классификация пары фрагментов в исходном BERT [3].

При обычном применении BERT подаем целиком известный текст и получаем его состояния. Для классификации рецензии маскировать слова не требуется. Если же хотим использовать предобученный MLM-классификатор для заполнения пропуска, подаем текст с `[MASK]` и смотрим распределение в этой позиции.

2.3. Next sentence prediction

В оригинальной модели была еще одна задача - next sentence prediction, или NSP. Берем два фрагмента A и B. В половине примеров B действительно следует за A в корпусе, в другой половине второй фрагмент выбирается случайно. Получается бинарная классификация, для которой метку можно построить из самого корпуса.

Пусть $r = 1$ означает настоящую соседнюю пару, $r = 0$ - случайную. По состоянию `[CLS]` классификатор выдает вероятность $q_\theta(A, B)$ класса 1. Потеря имеет вид

$$\mathcal{L}_{\text{NSP}} = -r \log q_\theta(A, B) - (1 - r) \log\bigl(1 - q_\theta(A, B)\bigr).$$

Если $r = 1$, остается первый член; если $r = 0$ - второй. Выходной слой и кодировщик обучаются совместно. Исходный BERT использует обе задачи предобучения: восстановление токенов и классификацию пары.

NSP не является определением всех BERT-подобных моделей. В RoBERTa эту задачу убрали, изменили режим маскирования и другие условия обучения [4]. Нам здесь достаточно этого примера: результат зависит не только от устройства блока трансформера, но и от того, какую обучающую задачу и данные мы выбрали.

2.4. Классификация текста и токенов

После предобучения состояние $h_{\text{CLS}} \in \mathbb{R}^d$ подается в классификатор. Для C классов:

$$p_{\theta,\varphi}(c \mid x) = \text{softmax}(W h_{\text{CLS}} + b)_c, \qquad W \in \mathbb{R}^{C \times d}, \quad b \in \mathbb{R}^C.$$

Параметры нового слоя обозначены $\varphi = (W, b)$. Для рецензии с правильным классом c^* минимизируем $-\log p_{\theta,\varphi}(c^* \mid x)$. Можно фиксировать BERT и обучать только φ, а можно обновлять и θ, и φ. Во втором случае представления подстраиваются под задачу тональности.

Для разметки текста одним состоянием уже не обойдемся. Например, требуется выделять имена людей, организации и места. Тогда над состоянием каждого размечаемого элемента ставим классификатор метки. Когда нужны ограничения на сочетания соседних меток, добавляем CRF. В приложении сохранен полный разбор этой задачи: энергия разметки, нормировочная константа, обучение и поиск согласованной последовательности. Это отдельная структура выхода поверх контекстных признаков BERT или другого кодировщика.

3. GPT: предсказание следующего токена

3.1. Авторегрессионная задача

В авторегрессионной постановке на входе находится начало текста, а на выходе требуется продолжение. Например, по префиксу «масло стекло на» модель строит распределение следующего токена. Правый контекст неизвестен: его еще предстоит сгенерировать.

Вероятность последовательности можно представить как произведение условных вероятностей:

$$p_\theta(x_1, \ldots, x_T) = \prod_{t=1}^{T} p_\theta(x_t \mid x_{<t}), \qquad x_{<t} = (x_1, \ldots, x_{t-1}).$$

Для первого предсказания префикс пуст. В учебной записи будем задавать его стартовым токеном $x_0 =$ `<bos>`. Последний целевой токен может быть специальным символом конца текста. Конкретная модель использует свои правила начала и завершения последовательности.

Такую задачу решают GPT-подобные модели: стек трансформерных блоков с причинным self-attention, без отдельного кодировщика входного текста и без cross-attention к нему [5, 6]. Название decoder-only относится к этому режиму работы. Сам блок не является полной копией декодировщика из схемы машинного перевода, где есть еще отдельный кодировщик и обращение к его состояниям.

3.2. Причинная маска и сдвиг целей

В тренировочном примере весь текст нам известен. Но использовать будущие токены для текущего предсказания нельзя. Если модель увидит правильный ответ через внимание, задача станет другой.

Поэтому позиция i обращается только к позициям $j \le i$. В формуле внимания прибавляем маску

$$M_{ij} = \begin{cases} 0, & j \le i, \\ -\infty, & j > i. \end{cases}$$

Индексы i и j обозначают позиции запроса и ключа. После softmax вес запрещенной связи равен нулю. Сама маска не обучается: она задает допустимые зависимости вычислительного графа.

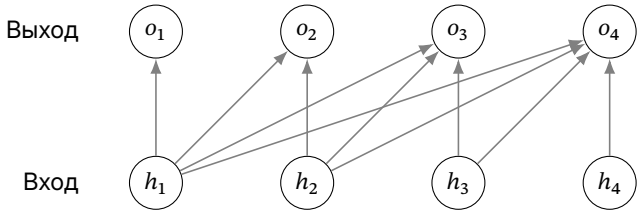


Рис. 3: Причинное self-attention: выходная позиция использует свою и предшествующие входные позиции.

Вход и правильные ответы сдвинуты на одну позицию:

вход	<bos>	x_1	x_2	x_3
цель	x_1	x_2	x_3	x_4

Состояние стартового токена предсказывает x_1, состояние после чтения x_1 предсказывает x_2 и так далее. Поэтому разрешенная диагональ причинной маски не открывает правильный ответ: цель сдвинута относительно входа на одну позицию.

Пусть h_{t-1} - состояние позиции, в которой прочитан последний токен известного префикса. Выходной слой вычисляет распределение вероятностей следующего токена:

$$p_\theta(x_t = a \mid x_{<t}) = \mathrm{softmax}(Uh_{t-1} + b)_a, \qquad U \in \mathbb{R}^{V \times d}, \quad b \in \mathbb{R}^V.$$

Здесь a - возможный токен словаря. Обучаются трансформер, входные эмбеддинги и выходной слой; в конкретной реализации входная и выходная таблицы могут иметь общие веса. Логарифм произведения превращается в сумму, и отрицательное логарифмическое правдоподобие равно

$$\mathcal{L}_{\mathrm{GPT}}(x) = -\sum_{t=1}^{T} \log p_\theta(x_t \mid x_{<t}).$$

Правильные ответы берутся прямо из текста. Для каждого t сеть получает предшествующие токены исходного текста. Все позиции одного обучающего примера можно обработать параллельно: маска уже запрещает связи с будущим. Дальше складываем потери и делаем обычный шаг оптимизации.

3.3. Генерация и примеры в контексте

При генерации правильного продолжения нет. Подаем начальный текст, получаем распределение следующего токена, выбираем токен и добавляем его к префиксу. Затем повторяем тот же шаг. Останавливаемся по символу конца или по ограничению длины. Таким образом, обучение использует известные предыдущие токены, а генерация - уже выбранные токены самой модели.

Выбор токена тоже влияет на ответ. Жадный метод каждый раз берет самый вероятный токен, семплирование выбирает случайно из распределения. Beam search хранит несколько возможных продолжений.

Генерацию можно использовать и для классификации. Допустим, мы хотим получить после рецензии слово «положительная» или «отрицательная». Тогда правильный выход задается текстом, а модель дописывает метку тональности после рецензии. Однако способность предсказывать продолжение сама по себе не гарантирует, что модель выполнит любую инструкцию.

В GPT-3 исследовали применение модели с примерами задачи прямо во входном контексте [7]. Например:

> Рецензия: отличный фильм. Тональность: положительная.
> Рецензия: скучный фильм. Тональность: отрицательная.
> Рецензия: прекрасная актерская игра. Тональность:

После последней строки модель должна продолжить текст меткой новой рецензии. Этот режим называют in-context learning. Здесь параметры θ фиксированы: примеры меняют вход и скрытые состояния, но шагов оптимизации по этим примерам нет. Это отличается от дообучения BERT на размеченной выборке, где параметры обновлялись.

Без демонстрационных примеров говорят о zero-shot, с одним примером - о one-shot, с несколькими - о few-shot. В данном случае речь именно о демонстрациях во входе. Их полезность проверяем на новых примерах: порядок, формулировки и состав контекста могут менять результат.

4. T5: условная генерация текста

4.1. Формат text-to-text

В T5 заданы входной и отдельный выходной текст. Например, входом служит статья, а выходом - краткое содержание. Для перевода входом становится предложение на исходном языке, а выходом - перевод. В классификации выходом может быть текстовая запись класса. T5 приводит эти задачи к общему формату text-to-text [8].

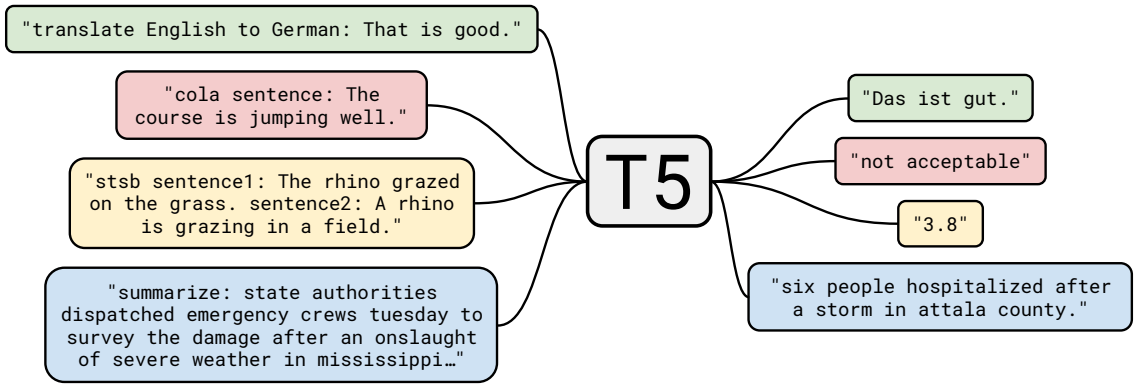


Рис. 4: Примеры задач в формате text-to-text: перевод, классификация, оценка сходства и суммаризация. Raffel et al., рис. 1 [8].

Для этого используются обе части трансформера. Кодировщик обрабатывает весь известный вход. Декодировщик порождает ответ слева направо. В его self-attention стоит причинная маска, а cross-attention обращается к состояниям входа: запросы приходят из декодировщика, ключи и значения - из кодировщика. Порядок токенов в T5 учитывается через обучаемые смещения оценок внимания, зависящие от

относительных позиций. Поэтому складывать вход с абсолютной позиционной таблицей BERT здесь не требуется.

Например, для тональности вход может начинаться с префикса `sst2:`, после которого стоит рецензия, а выходом будет слово `positive` или `negative`. Для перевода используется префикс вида `translate English to German:`. Префикс обозначает задачу в обучающих примерах. Его присутствие имеет смысл в связи с тем, на каких задачах и в каких форматах обучали конкретную модель.

Исходная статья T5 сравнивает разные архитектуры, задачи предобучения и способы переноса. Здесь будем разбирать одну основную схему: предобучение восстановлением скрытых фрагментов, затем дообучение на нужные пары входных и выходных текстов. Для самого предобучения вручную размеченные переводы или классы не нужны: пару можно построить из одного обычного текста.

4.2. Восстановление скрытых фрагментов

В BERT мы выбирали отдельные позиции и предсказывали исходные токены в этих позициях. В T5 построим целую выходную последовательность из скрытых фрагментов. Возьмем текст и заменим несколько непрерывных фрагментов специальными маркерами. Каждый скрытый фрагмент получает свой маркер, который еще называют sentinel-токеном.

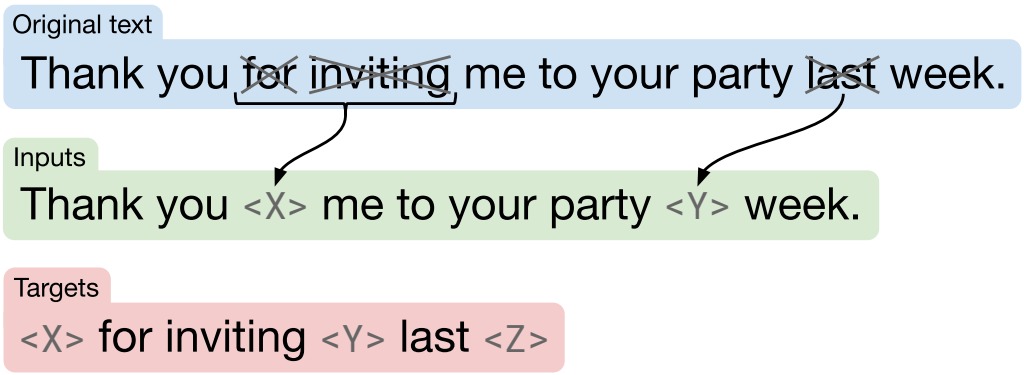


Рис. 5: Восстановление скрытых фрагментов в T5. Маркеры <X>, <Y> и <Z> обозначают разные sentinel-токены. Raffel et al., рис. 2 [8].

В следующем примере слова для наглядности считаются отдельными токенами:

> Исходный текст: Сегодня мы изучаем языковые модели на лекции.
>
> Вход кодировщика: Сегодня <extra_id_0> модели <extra_id_1>.
>
> Правильный выход: <extra_id_0> мы изучаем языковые <extra_id_1> на лекции <extra_id_2>

Первый маркер заменил фрагмент «мы изучаем языковые», второй - «на лекции». В целевой последовательности каждый маркер стоит перед содержимым соответствующего пропуска. Последний маркер закрывает последний скрытый фрагмент. В последовательность ответа также добавляется предусмотренный моделью токен конца.

По этим маркерам можно сопоставить части выхода с пропусками входа. Один общий `[MASK]` такого соответствия не задает. Модель должна породить и сами маркеры, и скрытые токены; открытые фрагменты входа в целевой текст не переписываются. Поэтому длина цели обычно меньше длины исходного текста.

Обозначим исходный текст через x, измененный вход через $\tilde{x} = c(x)$, целевой текст из скрытых фрагментов через $y = g(x)$. Для построения входа $c(x)$ и цели $g(x)$ выбираются одни и те же скрытые фрагменты. В итоговой схеме исходной статьи скрывают около 15% токенов со средней длиной фрагмента 3. Эти числа задают способ построения обучающих примеров; они не являются параметрами, которые сеть подбирает градиентным методом.

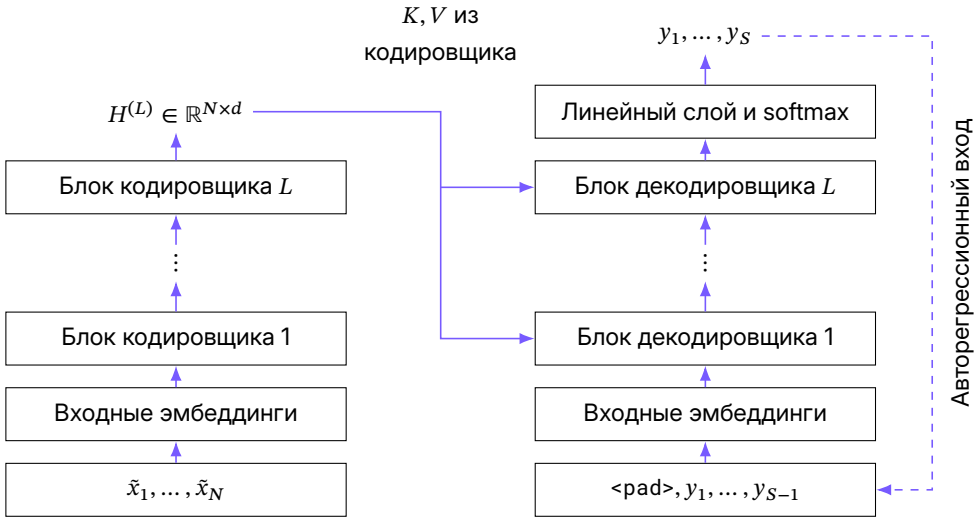


Рис. 6: Кодировщик обрабатывает измененный вход, декодировщик восстанавливает последовательность скрытых фрагментов. Схема T5 [8].

4.3. Функция потерь и применение

Пусть целевая последовательность имеет длину S. Ее вероятность факторизуется так же, как у авторегрессионной модели, но каждый шаг дополнительно обусловлен входом кодировщика:

$$p_\theta(y \mid \tilde{x}) = \prod_{t=1}^{S} p_\theta(y_t \mid y_{<t}, \tilde{x}).$$

Здесь $y_{<t}$ - уже известный префикс целевого текста. Кодировщик видит всю последовательность $\tilde{x}$, а декодировщик при предсказании y_t видит только предыдущие токены цели. Поэтому двунаправленный вход и причинный выход не противоречат друг другу.

На обучении используем правильный префикс $y^*_{<t}$. Потеря равна

$$\mathcal{L}_{\text{T5}}(x) = -\sum_{t=1}^{S} \log p_\theta(y^*_t \mid y^*_{<t}, c(x)).$$

Сумма идет по целевой последовательности, включая маркеры и токен конца, но исключая заполнение батча. Градиент обучает выходной слой, декодировщик, cross-attention, кодировщик и эмбеддинги. Вход декодировщика сдвинут относительно цели: стартовый символ, затем правильные предыдущие токены. В T5 роль стартового символа выполняет <pad>; здесь он используется для запуска декодировщика, а не как размеченная целевая позиция.

Подача правильных предыдущих токенов называется teacher forcing. Она позволяет параллельно считать все позиции цели при обучении. При генерации подаем стартовый символ, затем уже выбранные токены модели. Ошибка в начале ответа меняет контекст следующих шагов.

При дообучении на перевод вместо $c(x)$ подаем исходное предложение с префиксом задачи, а вместо $g(x)$ используем правильный перевод. Для суммаризации это статья и ее краткое содержание. Форма

условной кросс-энтропии остается той же, но правильные ответы теперь берутся из размеченной выборки задачи. На инференсе перевод или краткое содержание заранее неизвестны, и декодировщик строит их последовательно.

5. Сравнение моделей и оценка качества

Основные различия приведены в таблице. Во всех случаях предсказываются токены, но обучающие примеры устроены по-разному.

	BERT	**GPT**	**T5**
Архитектура	Кодировщик	Причинный трансформер	Кодировщик и декодировщик
Вход при предобучении	Текст с измененными выбранными токенами	Предшествующие токены исходного текста	Текст со скрытыми фрагментами
Правильный ответ	Исходный токен выбранной позиции	Следующий токен	Последовательность скрытых фрагментов и маркеров
Доступ к контексту	С обеих сторон входной позиции	Только известный префикс	Весь вход и префикс ответа
Обычное применение	Признаки и выходной слой задачи	Продолжение входного текста	Отдельный выходной текст по входному

Для классификации рецензий возможны три схемы. BERT получает текст и передает состояние `[CLS]` классификатору. GPT продолжает заданный текст меткой тональности. T5 получает рецензию с обозначением задачи и порождает текст класса отдельным декодировщиком. Сравнивать качество нужно на одних и тех же новых рецензиях и по одному правилу оценки.

Для классификации можно считать accuracy или F1. Если классы несбалансированы, одна accuracy может скрывать плохую работу на редком классе. Для именованных сущностей оценивают совпадение целых фрагментов и их типов. Верная метка на большинстве отдельных токенов еще не означает верные границы сущностей.

Для генерации сначала разделим две оценки. Отрицательное логарифмическое правдоподобие показывает, какую вероятность модель присвоила правильному тексту. Прикладная метрика оценивает сгенерированный ответ. В переводе это могут быть BLEU и оценка человеком, в суммаризации - ROUGE и проверка содержания. Большая вероятность текста не гарантирует правильности изложенных в нем фактов.

Для GPT среднюю токенную потерю часто переводят в perplexity:

$$\mathrm{PPL} = \exp\left(-\frac{1}{N}\sum_{n=1}^{N}\log p_\theta(x_n \mid x_{<n})\right).$$

Здесь N - число оцениваемых целевых токенов в корпусе, а логарифм натуральный. Чем меньше средняя потеря, тем меньше perplexity. При сравнении нужны одинаковые данные, токенизация и правила учета токенов. Потерю MLM по скрытым позициям нельзя напрямую сравнивать с потерей GPT по всем очередным токенам: различаются и условия предсказания, и набор целей.

Ограничение длины входа относится ко всем трем моделям. Если документ не помещается в контекст, нужно выбрать способ обработки его частей. Полное self-attention связывает позиции напрямую, но строит пары между позициями: в обычной явной реализации матрица весов имеет T^2 элементов.

Настройки выбираем по валидационной выборке, а тест используем для окончательной оценки. Для few-shot режима демонстрационные примеры тоже должны быть отделены от оцениваемых ответов. Предобучение дает параметры, которые затем используются в прикладной задаче; пригодность модели для конкретной задачи подтверждается результатом этой задачи на отложенных данных.

A. Контекстные представления: полный разбор

Статические векторы не различают значения одного слова в разных контекстах. Например, «стекло» в выражении «оконное стекло» и «стекло» в предложении «масло стекло на пол» получит один и тот же словный вектор. Требуется представление отдельного употребления:

$$r_t = F_\theta(x_1, \dots, x_T)_t.$$

Здесь r_t относится к позиции t в данном тексте и может меняться при изменении других слов. Такую модель называют контекстной. Доступный контекст ограничен тем фрагментом, который подали модели; он не охватывает автоматически весь документ или всю информацию о мире.

Контекст полезен не только для многозначных слов. В предложении «Вася вернулся домой. Он был в плохом настроении» представления могут учитывать, что местоимение относится к человеку. Для окончательного решения задачи кореференции требуется еще обученный механизм выбора связи, но признаки уже могут содержать необходимую информацию.

A.1. CoVe: предобучение на машинном переводе

В CoVe контекстный кодировщик обучается в составе модели перевода с английского языка на немецкий [1]. На входе предложение $x_1, \dots, x_T$, правильный выход - перевод $y_1^*, \dots, y_S^*$. Кодировщик строит состояния $h_1, \dots, h_T$, а декодировщик по ним порождает перевод. Длины входа и выхода могут различаться.

Механизм внимания на шаге декодирования выбирает, какие состояния входа использовать. Если s_{t-1} - предыдущее состояние декодировщика, можно записать

$$\alpha_{tj} = \frac{\exp a(s_{t-1}, h_j)}{\sum_{k=1}^{T} \exp a(s_{t-1}, h_k)}, \qquad c_t = \sum_{j=1}^{T} \alpha_{tj} h_j.$$

Функция a оценивает совместимость состояния декодировщика и состояния входа, а c_t - контекстный вектор для генерации очередного слова. Обучаемая функция совместимости или проекции позволяют согласовать размерности двух состояний.

Если перевод требует различать значения слов, кодировщик получает соответствующий обучающий сигнал. После обучения его скрытые состояния можно использовать как контекстные представления в другой задаче. В CoVe эту роль выполняет двунаправленный LSTM-кодировщик.

Ограничение метода связано с данными: нужны пары предложений с переводами. Таких примеров меньше, чем произвольных текстов без разметки. Поэтому следующий подход сохраняет идею контекстных признаков, но использует задачу, для которой правильный ответ можно получить из самого текста.

A.2. ELMo: предобучение языковой модели

Языковая модель оценивает вероятность текста через предсказание очередного токена по уже известным:

$$p(x_1, \dots, x_T) = \prod_{t=1}^{T} p(x_t \mid x_{<t}), \qquad x_{<t} = (x_1, \dots, x_{t-1}).$$

Пустой контекст для первого токена заменяется начальным символом. Правильный ответ на каждом шаге - следующий токен корпуса, поэтому отдельная ручная разметка не нужна.

Нельзя предсказывать x_t из состояния, которое уже видело сам x_t, и считать это изучением контекста. У модели появится возможность напрямую скопировать ответ. Для корректной задачи предсказания следующий токен должен быть исключен из доступного входа.

В ELMo обучают две направленные языковые модели [2]. Одна предсказывает токен по левому контексту, другая - по правому. Их суммарный логарифм правдоподобия равен

$$J_{\text{biLM}} = \sum_{t=1}^{T} \left[\log p_{\rightarrow}(x_t \mid x_{<t}) + \log p_{\leftarrow}(x_t \mid x_{>t}) \right].$$

Каждое направленное предсказание использует состояние до чтения целевого токена в своем направлении. Внутри предобучения скрытые состояния противоположных направлений не смешиваются так, чтобы правильный ответ проник в контекст прогноза. Это отличается от произвольного многослойного BiLSTM, в котором на следующих слоях можно обмениваться информацией между направлениями.

После предобучения получаем представления токена из обоих направлений на разных слоях. Пусть L - число рекуррентных слоев, $h_{t,0}$ - входное представление, а

$$h_{t,\ell} = [\vec{h}_{t,\ell}; \overleftarrow{h}_{t,\ell}], \qquad \ell = 1, \dots, L,$$

где квадратные скобки обозначают конкатенацию. В исходном ELMo входное представление строится из символов сверточной сетью. Это дает возможность вычислять признаки и для слов, которых не было в словаре обучения. При построении смеси все $h_{t,\ell}$ имеют согласованную размерность.

Самый простой способ переноса - взять последний слой. Более гибкий способ - обучить смесь слоев для конкретной задачи $\mathcal{T}$:

$$r_t^{\mathcal{T}} = \gamma_{\mathcal{T}} \sum_{\ell=0}^{L} s_\ell^{\mathcal{T}} h_{t,\ell}, \qquad s_\ell^{\mathcal{T}} = \frac{\exp a_\ell^{\mathcal{T}}}{\sum_{j=0}^{L} \exp a_j^{\mathcal{T}}}.$$

Softmax обеспечивает неотрицательные веса с суммой один. Параметры $a_\ell^{\mathcal{T}}$ определяют вклад каждого слоя, а скаляр $\gamma_{\mathcal{T}}$ - общий масштаб представления. Их обучают по функции потерь целевой задачи. Предобученную языковую модель можно при этом фиксировать. Разные задачи получают разные смеси.

Нижние и верхние слои могут содержать разную информацию о синтаксисе и значении. Однако слой не имеет заранее установленной единственной функции: полезность проверяется экспериментально. Существенное отличие от статической таблицы состоит в том, что итоговое представление зависит от употребления токена.

B. Разметка последовательности и CRF

В этом приложении d_h обозначает размер состояния кодировщика, то есть ту же величину, которую в основном тексте мы обозначали через d. Буква E в формуле энергии обозначает скалярную оценку полной разметки, а не таблицу входных эмбеддингов.

B.1. Разметка последовательности и именованные сущности

Теперь вернемся к тексту и рассмотрим другой тип выхода. В классификации один класс относился ко всему тексту. В переводе сеть порождала новую последовательность собственной длины. В задаче разметки каждому входному элементу требуется назначить метку:

$$x = (x_1, \dots, x_T) \quad \longmapsto \quad y = (y_1, \dots, y_T).$$

Примеры - определение части речи для каждого слова и распознавание именованных сущностей, NER. В NER требуется выделить фрагменты, которые обозначают человека, организацию, место или сущность другого заданного типа. Правильный ответ включает границы и тип каждого фрагмента. Для NER качество обычно измеряют по совпадению сущностей целиком: верны ли одновременно границы и тип. Метрики precision, recall и F1 считают по извлеченным сущностям, поэтому точность отдельных токенов может давать неполную картину.

Сначала токенизатор и кодировщик дают скрытые состояния h_t. Простейшая модель применяет линейный классификатор к каждому состоянию:

$$p_\theta(y_t = k \mid x) = \mathrm{softmax}(Wh_t + b)_k, \qquad \mathcal{L}_{\text{token}} = -\sum_{t=1}^{T} \log p_\theta(y_t^* \mid x).$$

Матрица $W \in \mathbb{R}^{K \times d_h}$ и смещение $b \in \mathbb{R}^K$ обучаются, K - число меток. Кодировщик может быть двунаправленной рекуррентной сетью или трансформером. Он учитывает контекст, поэтому признаки разных позиций уже связаны между собой.

Тем не менее распределение выходных меток в этой модели факторизуется:

$$p_\theta(y \mid x) = \prod_{t=1}^{T} p_\theta(y_t \mid x).$$

После получения состояний каждую метку выбирают отдельно. Модель может назвать начало сущности в одной позиции и поставить несовместимое продолжение в следующей. Контекстность признаков и явная согласованность выходных меток - разные свойства.

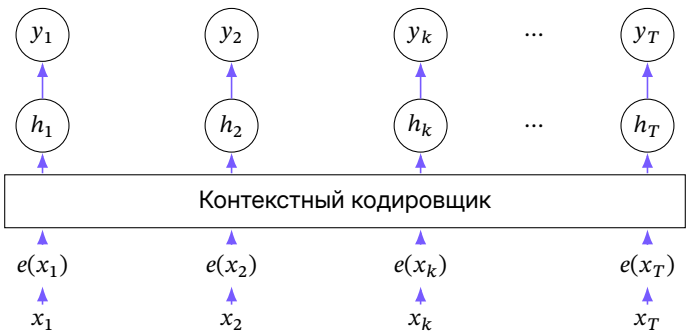


Рис. 7: Независимый выбор меток по контекстным признакам токенов.

Для границ сущностей используют специальные схемы разметки. В схеме BIOES метка B обозначает начало сущности, I - ее внутреннюю позицию, E - конец, S - сущность из одного токена, O - отсутствие сущности. К B, I, E и S добавляют тип, например PER, ORG или LOC.

Если последовательность меток содержит `B-LOC`, следующей должна быть `I-LOC` или `E-LOC`. Переход сразу к O оставил бы сущность незавершенной. В схеме BIO правила другие: отдельной метки конца нет, и переход от B к O может быть допустим для однотокенной сущности. Поэтому ограничения нужно задавать для выбранной схемы, а не для слов «начало» и «конец» вообще.

Если кодировщик использует подсловные токены, а разметка задана по словам, нужно согласовать эти уровни. Например, можно получать один вектор слова по его подсловным состояниям, а затем строить цепочку словных меток. Нельзя автоматически приравнивать число входных подслов к числу размеченных слов. Далее считаем, что такое согласование уже выполнено и позиция t соответствует одному размечаемому элементу.

B.2. Энергетическая модель и CRF

Чтобы учитывать совместимость соседних меток, будем оценивать всю последовательность. Для входа x зададим энергию

$$E_\theta(y, x) = \sum_{t=1}^{T} \phi_{\theta,t}(y_t, x) + \sum_{t=1}^{T-1} \psi_{\theta,t}(y_t, y_{t+1}, x).$$

Унарный потенциал $\phi_{\theta,t}$ оценивает метку в одной позиции, парный $\psi_{\theta,t}$ - соседние метки. В нашем соглашении меньшая энергия соответствует более предпочтительному ответу. Потенциалы могут иметь любые вещественные значения и сами по себе не являются вероятностями.

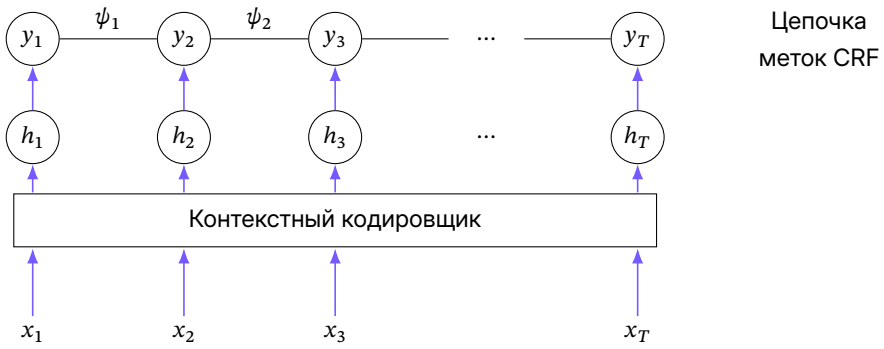


Рис. 8: Цепочка CRF добавляет совместимость соседних выходных меток.

Унарные потенциалы можно получить из выхода нейросети, взяв отрицательные оценки классов. Парные потенциалы можно хранить в обучаемой матрице переходов $K \times K$, общей для всех позиций. Более общая версия допускает зависимость перехода от позиции и входа. Даже постоянная матрица переходов уже явно связывает соседние выходы.

Условное распределение по последовательностям равно

$$p_\theta(y \mid x) = \frac{\exp(-E_\theta(y, x))}{Z_\theta(x)}, \qquad Z_\theta(x) = \sum_{y' \in \mathcal{Y}^T} \exp(-E_\theta(y', x)).$$

Здесь $\mathcal{Y}$ - множество K меток, а $Z_\theta(x)$ - нормировочная константа. Она суммирует веса всех возможных разметок и делает их вероятности суммирующимися в единицу. Такое условное распределение с цепочкой связей называется linear-chain CRF [11].

В общем случае у энергетической модели могут быть связи между произвольными выходами. Здесь выбрана цепочка: y_t связан только с y_{t-1} и y_{t+1}. При этом унарный потенциал может учитывать весь вход через двунаправленный кодировщик. Цепочка ограничивает структуру зависимости выходных меток, а не то, какую часть текста видит нейросеть.

Ограничения BIOES можно включить в парные потенциалы. Если переход запрещен, задаем ему $+\infty$, тогда его вес $\exp(-\psi)$ равен нулю. При разрешенном переходе используем конечную обучаемую оценку. Большой конечный штраф лишь сильно уменьшает вероятность и не дает строгого запрета. Для полного ограничения нужны также допустимые стартовые и конечные метки. Например, I не может начинать корректную последовательность, а B не может завершать ее в BIOES. Эти условия можно включить в потенциалы первой и последней позиций.

В упрощенном примере используются метки `<entity start>`, `<entity>`, `<entity end>` и `<no entity>`. Они обозначают начало, внутреннюю позицию, конец и отсутствие сущности. Это иллюстрация границ без разделения по типам и без отдельной метки однословной сущности. Пример конечного штрафа:

$$\psi_t(\texttt{<entity start>}, \texttt{<no entity>}, x) = 1000.$$

Такой переход получает множитель e^{-1000} и практически подавляется. Математически его вероятность остается положительной при конечных остальных потенциалах; строгому запрету соответствует бесконечная энергия.

Тем самым вероятность разметки определяется не только качеством каждой метки отдельно, но и ее совместимостью с соседними метками. На практике не все переходы задаются вручную: структурно невозможные исключают, а предпочтения между допустимыми переходами обучают по размеченным текстам.

B.3. Обучение CRF и нормировочная константа

Пусть y^* - правильная последовательность меток. Максимизация ее условного правдоподобия эквивалентна минимизации

$$\mathcal{L}_{\text{CRF}}(x, y^*) = -\log p_\theta(y^* \mid x) = E_\theta(y^*, x) + \log Z_\theta(x).$$

Первое слагаемое снижает энергию правильной разметки. Второе учитывает конкурирующие ответы. Если снижать только энергию правильного ответа без нормировки, нельзя корректно обучить условное распределение.

Для конечных обучаемых потенциалов, при фиксированных структурных запретах, градиент имеет вид

$$\nabla_\theta \mathcal{L}_{\text{CRF}} = \nabla_\theta E_\theta(y^*, x) + \nabla_\theta \log Z_\theta(x).$$

Распишем второе слагаемое:

$$\begin{aligned}\nabla_\theta \log Z_\theta(x) &= \frac{1}{Z_\theta(x)} \sum_y \exp(-E_\theta(y, x)) \, [-\nabla_\theta E_\theta(y, x)] \\ &= -\mathbb{E}_{y \sim p_\theta(\cdot|x)}[\nabla_\theta E_\theta(y, x)].\end{aligned}$$

Поэтому

$$\nabla_\theta \mathcal{L}_{\text{CRF}} = \nabla_\theta E_\theta(y^*, x) - \mathbb{E}_y[\nabla_\theta E_\theta(y, x)].$$

Обучение сопоставляет правильный ответ с распределением ответов текущей модели. В общем графе вычисление ожидания может быть сложным, и тогда используют приближенные методы, в том числе семплирование. Для цепочки сумма считается точно динамическим программированием.

По всей обучающей выборке $\mathcal{D}$ суммируем эти градиенты:

$$\nabla_\theta \mathcal{L}_\mathcal{D} = \sum_{(x,y^*)\in\mathcal{D}} \left[\nabla_\theta E_\theta(y^*, x) + \nabla_\theta \log Z_\theta(x)\right].$$

Если функция потерь усредняется по примерам, справа дополнительно стоит множитель $1/|\mathcal{D}|$. В стохастическом обучении полную сумму заменяют оценкой по текущему батчу.

Распишем нормировку, выделяя по одной метке слева направо. Для $T \geq 2$, опустив зависимость потенциалов от x и θ, получаем

$$\begin{aligned} Z(x) &= \sum_{y_1,\dots,y_T} \left(\prod_{t=1}^{T} e^{-\phi_t(y_t)}\right)\left(\prod_{t=1}^{T-1} e^{-\psi_t(y_t,y_{t+1})}\right) \\ &= \sum_{y_1} e^{-\phi_1(y_1)} \sum_{y_2} e^{-\psi_1(y_1,y_2)-\phi_2(y_2)} \dots \sum_{y_T} e^{-\psi_{T-1}(y_{T-1},y_T)-\phi_T(y_T)}. \end{aligned}$$

Вложенность существенна: внутренняя сумма зависит от граничной метки предыдущей позиции. Ее можно вычислить один раз для каждого возможного значения этой метки и повторно использовать.

Наивно Z содержит K^T слагаемых. Но каждый множитель зависит только от одного элемента или пары соседних. Для метки a в позиции t определим $B_t(a)$ как суммарный вес всех продолжений от этой позиции до конца при условии $y_t = a$, включая унарный множитель позиции t.

В последней позиции продолжение пусто:

$$B_T(a) = \exp(-\phi_T(a, x)).$$

Один шаг перед концом дает

$$B_{T-1}(a) = e^{-\phi_{T-1}(a,x)} \sum_{b\in\mathcal{Y}} e^{-\psi_{T-1}(a,b,x)} e^{-\phi_T(b,x)}.$$

Обозначение $B_t(a)$ позволяет отличать частичные суммы от полной нормировочной константы $Z(x)$.

Для остальных позиций выделим следующую метку b:

$$B_t(a) = \exp(-\phi_t(a, x)) \sum_{b\in\mathcal{Y}} \exp(-\psi_t(a, b, x)) B_{t+1}(b), \qquad t = T-1, \dots, 1.$$

Подстрочный индекс θ здесь опущен для краткости; потенциалы по-прежнему зависят от параметров. Формула перебирает только K вариантов следующей метки. Все дальнейшие продолжения уже учтены в $B_{t+1}(b)$. После прохода справа налево

$$Z_\theta(x) = \sum_{a\in\mathcal{Y}} B_1(a).$$

Для $T = 2$ запись раскрывается в знакомую сумму:

$$Z = \sum_a \sum_b \exp\left(-\phi_1(a, x) - \psi_1(a, b, x) - \phi_2(b, x)\right).$$

Это показывает, что рекурсия суммирует те же полные разметки, просто группирует их по общей границе между соседними позициями. Никакого приближения в этом преобразовании нет.

В каждом шаге считаем K значений $B_t(a)$, и для каждого суммируем K вариантов b. Сложность равна $O(TK^2)$. При нескольких десятках меток это существенно дешевле полного перебора. Для прямого расчета Z достаточно хранить два соседних вектора; обучение с обратным распространением может хранить промежуточные результаты.

Произведения экспонент могут численно исчезать или переполняться, поэтому расчет ведут в логарифмическом масштабе. Пусть $\ell_t(a) = \log B_t(a)$. Тогда

$$\begin{aligned}
\ell_T(a) &= -\phi_T(a, x), \\
\ell_t(a) &= -\phi_t(a, x) + \mathrm{LSE}_b \left(-\psi_t(a, b, x) + \ell_{t+1}(b)\right), \\
\log Z &= \mathrm{LSE}_a \, \ell_1(a),
\end{aligned}$$

где устойчивое log-sum-exp вычисляют как

$$\mathrm{LSE}(s_1, \dots, s_K) = m + \log \sum_{j=1}^{K} \exp(s_j - m), \qquad m = \max_j s_j.$$

Все операции для допустимых путей дифференцируемы. Можно провести обратное распространение через динамическую программу и обучить нейросеть и конечные параметры переходов. Эквивалентный расчет использует точные маргинальные вероятности состояний и переходов из алгоритма forward-backward. Семплирование для нормировки этой цепочки не требуется.

B.4. Выбор согласованной последовательности меток

На этапе применения нужна наиболее вероятная полная разметка. Для фиксированного x константа $Z_\theta(x)$ одинакова для всех ответов, поэтому

$$\hat{y} = \arg\max_y p_\theta(y \mid x) = \arg\min_y E_\theta(y, x).$$

Эта задача также решается динамическим программированием. Вместо суммы весов выбираем минимальную накопленную энергию. Пусть $D_t(a)$ - энергия лучшего продолжения от позиции t с меткой a. Тогда

$$\begin{aligned}
D_T(a) &= \phi_T(a, x), \\
D_t(a) &= \phi_t(a, x) + \min_{b \in \mathcal{Y}} \left[\psi_t(a, b, x) + D_{t+1}(b)\right].
\end{aligned}$$

В каждом шаге сохраняем метку b, на которой достигнут минимум. Выбираем лучшую начальную метку, затем по сохраненным решениям восстанавливаем весь путь. Это форма алгоритма Витерби. Его сложность также равна $O(TK^2)$, а поиск дает точный максимум для заданной цепочной модели.

Расчет нормировки при обучении и поиск ответа используют похожие рекурсии, но выполняют разные операции. Для Z нужны суммы по всем допустимым разметкам. Для наиболее вероятного ответа нужен один лучший путь. Нельзя заменить сумму максимумом в функции правдоподобия и считать, что обучение осталось тем же.

В результате у модели разметки есть два уровня. Кодировщик получает контекстные признаки каждого слова. CRF согласует соседние выходные метки и позволяет находить допустимую разметку всего текста. Если переходы не учитываются, остается независимый токенный классификатор. Если переходы нужны, цепочная структура дает точное и вычислительно доступное обучение.

Распознавание текста с изображения и распознавание речи требуют еще одной постановки: длина наблюдений обычно не равна длине выходной строки, и соответствие между ними заранее неизвестно. Разобранная здесь разметка предполагает, что одному размечаемому элементу соответствует одна выходная метка.

C. Маскированное обучение на изображениях

Идею маскированного предобучения можно перенести на изображения. Дано изображение без метки класса. Его разбивают на патчи и рассматривают их как последовательность. Каждый патч преобразуют в вектор, добавляют информацию о расположении и подают в трансформер. Часть патчей скрывают, а модель восстанавливает информацию о них по видимой части изображения.

Вместо предсказания каждого пикселя можно использовать дискретные визуальные коды. Отдельный обученный токенизатор переводит изображение в сетку индексов кодовой книги. Обозначим целевой индекс позиции i через $z_i \in \{1, \dots, K_{\text{img}}\}$. Тогда задача похожа на MLM:

$$\mathcal{L}_{\text{image}} = -\sum_{i \in \mathcal{M}} \log p_\theta(z_i \mid \tilde{I}),$$

где $\tilde{I}$ - изображение со скрытыми патчами, а $\mathcal{M}$ - выбранные позиции. Целевые коды дает визуальный токенизатор; трансформер и классификатор обучаются их предсказывать. На этапе извлечения признаков маски не нужны.

Такой механизм предобучения реализован, например, в BEiT [10]. В исходном BEiT использован дискретный вариационный автокодировщик, dVAE. Его следует отличать от VQ-VAE, где квантование выбирает ближайший вектор кодовой книги. Токенизатор задает дискретные целевые коды для предсказания скрытых патчей.

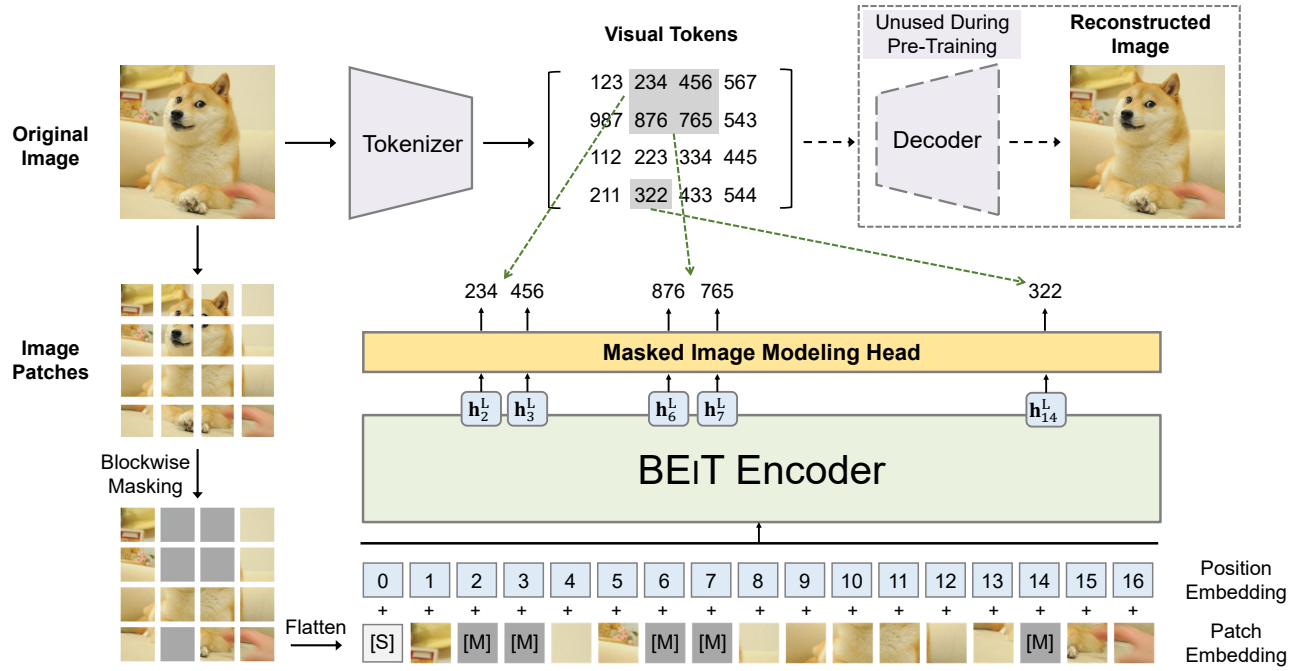


Рис. 9: Предобучение BEiT: токенизатор дает целевые визуальные коды, а трансформер предсказывает их для скрытых патчей. Декодер автокодировщика на этом этапе не используется. Bao et al., рис. 1 [10].

Дискретный автокодировщик может выдавать целый набор индексов для одного участка:

$$z(x_i) = (z_{i,1}, \dots, z_{i,d_{\text{img}}}) \in \{1, \dots, K_{\text{img}}\}^{d_{\text{img}}}.$$

Здесь d_{img} - число дискретных координат, каждая из которых выбирается из K_{img} значений. Для такой цели суммируют потери по скрытым участкам и координатам:

$$\mathcal{L} = -\sum_{i \in \mathcal{M}} \sum_{r=1}^{d_{\text{img}}} \log p_\theta(z_{i,r} \mid \tilde{I}).$$

Это обобщение записи с одним индексом на патч. Количество комбинаций кодов может быть равно $K_{\text{img}}^{d_{\text{img}}}$; оно не означает, что для каждой комбинации обучается отдельный вектор кодовой книги. Автокодировщик учится восстанавливать изображение через дискретное промежуточное представление, а маскированный трансформер использует это представление как целевой ответ.

Сам трансформер для изображений и способ его предобучения - разные решения. Из факта разбиения на патчи не следует, что модель обучена без разметки. Маскированное обучение дает такой сигнал из самого изображения. После предобучения признаки можно применять к классификации и другим задачам; улучшение по сравнению со сверточной сетью является результатом конкретного эксперимента, а не гарантией для любой задачи.

Литература

[1] McCann B. et al. Learned in Translation: Contextualized Word Vectors. 2017. arXiv:1708.00107.

[2] Peters M. E. et al. Deep Contextualized Word Representations. 2018. arXiv:1802.05365.

[3] Devlin J. et al. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. 2019. arXiv:1810.04805.

[4] Liu Y. et al. RoBERTa: A Robustly Optimized BERT Pretraining Approach. 2019. arXiv:1907.11692.

[5] Radford A. et al. Improving Language Understanding by Generative Pre-Training. 2018. Статья.

[6] Radford A. et al. Language Models are Unsupervised Multitask Learners. 2019. Статья.

[7] Brown T. B. et al. Language Models are Few-Shot Learners. 2020. arXiv:2005.14165.

[8] Raffel C. et al. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. 2020. arXiv:1910.10683.

[9] Vaswani A. et al. Attention Is All You Need. 2017. arXiv:1706.03762.

[10] Bao H. et al. BEiT: BERT Pre-Training of Image Transformers. 2022. arXiv:2106.08254.

[11] Sutton C., McCallum A. An Introduction to Conditional Random Fields. 2012. arXiv:1011.4088.